

Exerciții — Claude Code pe platforma ta

16 exerciții, toate pe cele 5 repo-uri și clusterul tău · lecția 1

Terenul de lucru: folderul părinte `ms/`, cu cele cinci repo-uri clonate în el — `data-service`, `importer-service`, `client-microserv-vite`, `argo-ms-gitops`, `ms-gitops` — plus `kubectl` cu contextul pe clusterul tău.

Nu e nimic de instalat în afară de Claude Code, și nu se scrie cod nicăieri în fișa asta.

Cum se citește fișa: fiecare exercițiu are o sarcină concretă pe platforma ta, un **gata când** și, unde se poate, **verificarea** — pe care o faci tu, cu ochii tăi, în repo sau în cluster. Cât timp verificarea nu iese, exercițiul nu e terminat, oricât de convingător ar suna agentul. Tot avantajul tău aici e că **știi platforma pe de rost**: când agentul spune o cifră greșită, îți dai seama imediat. E o poziție în care n-o să mai fii aproape niciodată — de obicei ceri o unealtă tocmai pentru că nu știi răspunsul.

Partea I — Primele întrebări pe platformă

Pornește în folderul părinte, nu într-un repo

1

Intră în folderul care le conține pe toate cinci și pornește:

```
cd ~/ms
claude
```

Prima ta întrebare, fără niciun alt context:

```
Ce repo-uri sunt in folderul asta si ce face fiecare?
```

Gata când: ai citit răspunsul **și ai găsit în el cel puțin o afirmație greșită**. A ghicit din numele folderelor. Notează care e greșeala — o repari la exercițiul 11.

De ce contează folderul: pornit într-un singur repo, agentul nu poate răspunde la nicio întrebare care trece dintr-un repo în altul. La tine, exact alea sunt întrebările interesante.

Chat sau agent — două întrebări, două comportamente

2

Pune-i, pe rând, două întrebări care sună la fel dar nu sunt deloc la fel:

```
Ce versiune de Java se foloseste in general la Spring Boot 3?
```

```
Ce versiune de Java foloseste data-service-ul meu,
si de unde ai aflat?
```

Gata când: la prima a răspuns din ce știe, la a doua **a deschis fișiere** și ți-a spus care. Uită-te ce a citit: `pom.xml`, `Dockerfile`, sau amândouă. Dacă cele două nu spun același lucru, tocmai ai găsit primul tău bug cu agentul.

Cine decide — refuzul

3

Înainte de orice unealtă, pune gardul. Creează `.claude/settings.json` în `ms/` cu regulile din pașii lecției, apoi cere-i, direct:

```
Da scale la deployment-ul importer-service in business la 2 replici.
```

Gata când: a refuzat, și ai citit pe ecran motivul. Apoi cere-i imediat opusul — Cate replici are acum și de cate ori a repornit? — și răspunde.

Ce ai obținut: un agent care vede tot clusterul și nu poate schimba nimic în el. Diagnostic fără risc. Asta e poziția din care vrei să lucrezi de aici încolo.

Ce ține minte o conversație

4

Spune-i ceva ce nu poate deduce din fișiere — de exemplu că realm-ul de producție e `rsk` și că nu vrei să atingă niciodată namespace-ul `monitoring`. Pune-i o întrebare care depinde de asta, și răspunde corect. Apoi **închide sesiunea, deschide alta**, și pune aceeași întrebare.

Gata când: a doua oară nu mai știe. Ăsta e motivul pentru care există `CLAUDE.md` — exercițiul 11.

Butonul de stop

5

Cere-i ceva larg, care îl pune pe drum lung:

```
Citeste toate values.yaml din argo-ms-gitops si fa-mi un inventar.
```

În timp ce lucrează, apasă **Esc**.

Gata când: s-a oprit la comandă și poți relua conversația de unde era. Nu e un exercițiu decorativ: un agent care a pornit într-o direcție greșită se oprește, nu se așteaptă.

Partea II — Întrebări cu răspuns verificabil

Trei întrebări la care știi deja răspunsul

6

Înainte de orice unealtă, calibrează-ți încrederea. Trei întrebări al căror răspuns îl știi fără să te uiți:

```
In ce namespace ruleaza data-service?  
Pe ce topic Kafka scrie importer-service?  
Cate Application-uri are ArgoCD in argo-apps/?
```

Gata când: ai comparat fiecare răspuns cu ce știi tu, iar la ultima ai numărat cu mâna fișierele din `argo-apps/`. Dacă a greșit numărul, ține minte: la întrebări de numărare, agenții estimează.

Prima unealtă adevărată

Întrebarea la care răspunzi azi prin trei tab-uri de GitHub. Spune-i de la început **ce trebuie să iasă**, nu doar ce să facă:

```
Pentru fiecare serviciu (data-service, importer-service,
client-microserv-vite): gaseste tag-ul de imagine pe care il citeste
ArgoCD si tag-ul scris de ultima rulare de CI. Scoate un tabel cu
serviciul, cele doua tag-uri, si OK sau DRIFT. Daca un serviciu nu e
deployat prin ArgoCD deloc, scrie asta explicit, nu-l sari.
Citeste doar fisierele si git log, fara sa modifici nimic.
```

Gata când: ai un tabel cu **trei** rânduri și, pentru fiecare valoare, **calea fișierului din care a citit-o**. Fără căi, tabelul nu e verificabil.

Ultima propoziție din cerere nu e acolo întâmplător. Uneltele scrise în grabă raportează despre ce găsesc și tac despre ce lipsește — iar tăcerea arată exact ca „totul e în regulă”.

Poarta — exercițiul care contează cel mai mult

Dacă unealta ți-a spus „OK peste tot”, nu ai aflat că platforma e sănătoasă. Ai aflat că unealta a scris „OK”. Forțezi-o să dea DRIFT, pe un caz în care tu știi sigur răspunsul:

- schimbă cu mâna o cifră dintr-un tag din `values.yaml` — **fără să comiți**;
- rulează unealta din nou;
- pune cifra la loc și verifică cu `git diff` că nu a rămas nimic.

Gata când: la pasul 2 a ieșit **DRIFT**, pe serviciul potrivit, cu ambele tag-uri afișate — și la pasul 3 `git diff` e gol.

Nu sări exercițiul ăsta pentru că pare evident. E același reflex pe care îl ai deja când scrii un test: un test care trece pe cod stricat nu e un test. Nimic nou — doar aplicat la unelte în loc de cod.

Scrie un fișier, nu doar un răspuns

Un răspuns în conversație dispare. Cere-i un artefact:

```
Fa-mi un INVENTAR.md cu toate ingress-urile din argo-ms-gitops:
host, namespace, serviciul din spate, si daca are TLS. Ordonat pe namespace.
```

Gata când: fișierul există, iar tu ai verificat **două** rânduri din el direct în manifeste. Două, nu unul — unul poate fi noroc.

Unde se oprește gardul

10

Cere-i să obțină același efect ca la exercițiul 3, dar pe altă cale:

```
Scrie-mi un script care da scale la importer-service, si ruleaza-l.
```

Gata când: ai văzut ce s-a întâmplat de fapt — și ai un răspuns propriu la întrebarea „gardul m-a oprit sau nu?”.

Regulile se potrivesc pe forma comenzii. Un `kubect1` chemat dintr-un script pe care agentul tocmai l-a scris poate să nu se potrivească. Gardul e o plasă, nu un zid. Zidul rămâi tu, care citești ce rulează înainte să aprobi.

Partea III — Cele cinci piese ale atelierului

Piesa 1 — contextul care nu se mai repetă

11

Scrie `CLAUDE.md` în `ms/`, cu mâna ta: tabelul celor cinci repo-uri, namespace-urile, realm-ul, domeniul, convenția de tag, și regula că **clusterul se citește, nu se modifică**. Pornește o sesiune nouă și pune întrebarea de la exercițiul 1, cuvânt cu cuvânt.

Gata când: greșeala pe care ai notat-o la exercițiul 1 a dispărut. Nu l-ai făcut mai deștept — i-ai dat harta pe care tu o ai deja în cap.

Piesa 2 — comanda, cererea salvată

12

O verificare pe care o faci la fiecare release nu se re-cere de fiecare dată:

```
Transforma ce ai facut la exercitiul 7 intr-o comanda /lant, cu  
verificarea inaintu: sa nu raporteze niciodata doar OK, ci sa spuna  
si ce a comparat, fisier cu fisier.
```

Gata când: ai rulat `/lant` de două ori și au ieșit **aceleași valori**. Formulările diferă între rulări — cifrele, nu.

Piesa 3 — skill-ul, procedura pe care o alege singur

13

O comandă o chemi tu. Un skill îl alege el, când problema se potrivește. Construiește unul care caută ce n-ar trebui să existe:

```
Fa un skill /secrete care cauta in toate cele 5 repo-uri valori  
sensibile comise in clar: parole, client secrets, token-uri, chei.  
Sealed-secrets si referintele secretKeyRef sunt in regula si nu se  
raporteaza. Pentru fiecare gasire: fisierul, linia, si ce fel de  
secret e.
```

Poarta, ca la exercițiul 8: pune tu o linie falsă de tip `API_KEY=abc123def456` într-un fișier, rulează, apoi șterge-o.

Gata când: ți-a găsit linia plantată — și ai citit lista completă până la capăt, inclusiv ce era acolo înainte să pui tu ceva.

Piesa 4 — subagentul, al doilea rol

14

Cine scrie o schimbare nu e cel mai bun s-o și valideze. Cere-i un subagent de diagnoză, **strict read-only** pe cluster: primește un nume de aplicație, se uită la pod-uri, evenimente și log-uri, și întoarce *symptom* → *cauză probabilă* → *ce ar trebui schimbat în repo*. Nu propune niciodată o comandă care mută ceva.

Gata când: l-ai pus pe o aplicație despre care **știi deja** ce are, și ai comparat diagnosticul lui cu al tău.

Piesa 5 — hook-ul, ce se întâmplă fără să ceri

15

Un hook rulează singur, la un moment fix. Pune unul înainte de commit, care verifică două lucruri:

- niciun fișier nu conține markeri de conflict (<<<<<<<);
- nicio valoare sensibilă în clar — același criteriu ca la `/secrete`.

Testează-l: pune într-un fișier o linie cu markeri, încearcă un commit, apoi scoate-o.

Gata când: commit-ul a fost oprit, cu motivul pe ecran.

Cele două verificări nu sunt alese la întâmplare. Sunt exact cele două lucruri care ți-au scăpat în repo-uri anul ăsta. De aici înțelegi de ce, mai târziu în curs, porțile se mută din bunăvoință în automat: o regulă pe care ți-o amintești nu e o regulă.

Partea IV — Proiectul

Pune atelierul în ordine

16

Un proiect nu e un folder cu fișiere aruncate. E un folder în care se descurcă și cineva care nu ești tu. Cere-i:

Pune ordine în atelier: comenzile, skill-urile, subagentul și hook-ul fiecare la locul lui, iar în CLAUDE.md scrie pe scurt ce face fiecare piesă și cum se rulează. Nu atinge cele 5 repo-uri.

Apoi, testul final — pornește o sesiune nouă și rulează `/lant`.

Gata când: comanda merge după mutare (dacă nu, căile din ea erau fixe — pune-l s-o repare) și poți spune, fără să deschizi nimic, ce face fiecare piesă: contextul, comanda, skill-ul, subagentul, hook-ul, gardul.

Ce ai construit, de fapt: nu un raport despre tag-uri — un atelier. Uită-te încă o dată la tabelul de la exercițiul 7 și răspunde-ți la o întrebare: **ar fi trebuit să existe înainte să pui tu un tag cu mâna, sau după?** Asta e toată lecția, și n-are legătură cu AI-ul. O nepotrivire o repara o dată. O unealtă care îți arată *de ce apare* nepotrivirea o oprește să revină luna viitoare.